API SECURITY RISK ANALYSIS REPORT

Future Interns – Cyber Security Internship | Task 3

Prepared By	Ahmar Shafith Baijur Ahamed
Internship Track	Cyber Security
Task	Task 3 – API Security Risk Analysis
API Tested	JSONPlaceholder (https://jsonplaceholder.typicode.com)
Date	May 2026
Repository	FUTURE_CS_03

1. Executive Summary

This report presents the findings of a read-only API Security Risk Analysis conducted on the JSONPlaceholder public test API (https://jsonplaceholder.typicode.com). The assessment was performed as part of the Future Interns Cyber Security Internship – Task 3.

The analysis identified six key security risks across multiple API endpoints including unauthenticated access, excessive data exposure, missing rate limiting, broken object-level authorization, missing security headers, and unrestricted write access. These findings reflect real-world API security concerns aligned with the OWASP API Security Top 10 framework.

All testing was conducted in a read-only, ethical, and non-exploitative manner using only publicly available demo API endpoints.

2. Scope & Ethics

2.1 Assessment Scope

The scope of this assessment was limited to the following publicly available JSONPlaceholder API endpoints:

- GET /users – Retrieve all users
- GET /users/1 – Retrieve a single user by ID
- GET /posts – Retrieve all posts
- GET /todos – Retrieve all to-do items
- POST /posts – Simulate resource creation

2.2 Ethical Boundaries

The following ethical guidelines were strictly followed throughout the assessment:

- Only read-only and safe demo API requests were used
- No exploitation or bypass attempts were made
- No DoS or flooding tests were performed
- No private or production APIs were targeted
- All findings are based on documentation analysis and response inspection only

3. Tools Used

Tool	Purpose
Postman	API endpoint testing, request sending, response inspection
Browser DevTools	Header inspection and network analysis
JSONPlaceholder API	Public demo REST API used as target for analysis

OWASP API Security Top 10	Industry framework for classifying API security risks
GitHub	Repository hosting for report and screenshots
MS Word / PDF	Professional report documentation

4. Methodology

The assessment followed a structured, documentation-first approach aligned with real-world API security auditing practices:

Step	Activity	Description
1	API Selection	Selected JSONPlaceholder as a publicly available, ethically appropriate test API
2	Documentation Review	Reviewed all available API endpoints and expected behaviors
3	Endpoint Testing	Sent GET and POST requests via Postman; recorded responses
4	Header Analysis	Inspected response headers for missing security controls
5	Risk Identification	Mapped findings to OWASP API Security Top 10 categories
6	Risk Classification	Assigned severity levels: Low, Medium, or High
7	Reporting	Documented all findings with business impact and remediation steps

5. Findings Summary

The following table summarizes all identified security risks:

#	Finding	Severity	OWASP Category	Endpoint
1	**Unauthenticated API Access**	**High**	API1:2023 – Broken Object Level Auth	/users, /posts, /todos
2	**Excessive Data Exposure**	**High**	API3:2023 – Excessive Data Exposure	/users, /users/1
3	**Missing Rate Limiting**	**High**	API4:2023 – Unrestricted Resource Consumption	/posts (100 records)

4	**Broken Object Level Authorization**	**Medium**	API1:2023 – BOLA	/users/1, /users/2...
5	**Missing Security Headers**	**Medium**	API8:2023 – Security Misconfiguration	All endpoints
6	**Unrestricted Write Access**	**Medium**	API2:2023 – Broken Auth	POST /posts

6. Detailed Findings

Finding 1: Unauthenticated API Access

Severity	**HIGH**
Endpoint(s)	/users | /posts | /todos
OWASP	API1:2023 – Broken Object Level Authorization
Observation	All API endpoints (GET /users, GET /posts, GET /todos) returned data with a 200 OK status code without requiring any authentication token, API key, or session credential.
Evidence	Screenshot Test1a – GET /users returns full user list with Status: 200 OK and no Authorization header required. Screenshot Test-3 – GET /posts returns 100 posts with no credentials.
Business Impact	In a production environment, this would allow any anonymous user or attacker to retrieve all customer records, posts, or user activity without logging in. This constitutes a critical data breach risk.
Remediation	Implement authentication on all API endpoints. Use OAuth 2.0 or JWT (JSON Web Tokens). Require a valid Bearer token in the Authorization header for all data-access endpoints.

Finding 2: Excessive Data Exposure

Severity	**HIGH**
Endpoint(s)	/users | /users/1
OWASP	API3:2023 – Excessive Data Exposure
Observation	The GET /users and GET /users/1 endpoints returned extensive personal data fields including: full name, username, email address, full street address, suite number, city, zipcode, geo-coordinates (latitude/longitude), phone number, website URL, and full company details.
Evidence	Screenshot Test1a – Response shows name, email, address, phone, geo-coordinates, company data. Screenshot Test-2 – GET /users/1 exposes identical PII fields for a single user.

Business Impact	APIs should return only the minimum data required for the requesting client. Returning email, phone, address, and geo-coordinates unnecessarily increases the risk of PII exposure, violates GDPR/data privacy regulations, and creates liability for the organization.
Remediation	Apply response filtering to return only required fields. Use Data Transfer Objects (DTOs) to shape API responses. Never expose internal fields (geo-coordinates, company internal data) unless explicitly needed.

Finding 3: Missing Rate Limiting

Severity	**HIGH**
Endpoint(s)	/posts (returned 100 records)
OWASP	API4:2023 – Unrestricted Resource Consumption
Observation	The GET /posts endpoint returned all 100 records in a single response (7.98 KB) with no pagination enforcement, no rate-limit headers (X-RateLimit-Limit, Retry-After), and no throttling observed. Repeated requests were fulfilled without restriction.
Evidence	Screenshot Test-3 – GET /posts returns 100 items, Status 200 OK, Size 7.98 KB. No rate-limit headers visible in response headers.
Business Impact	Without rate limiting, attackers can flood the API with thousands of requests per second, causing server overload, degraded performance for legitimate users, and potential Denial of Service (DoS). It also enables data harvesting at scale.
Remediation	Implement API rate limiting (e.g., 100 requests per minute per IP). Add X-RateLimit-Limit and X-RateLimit-Remaining headers to responses. Enforce pagination (e.g., limit=20&page=1) on all list endpoints.

Finding 4: Broken Object Level Authorization (BOLA)

Severity	**MEDIUM**
Endpoint(s)	/users/1, /users/2, /users/3 ...
OWASP	API1:2023 – Broken Object Level Authorization
Observation	The API exposes individual user records via sequential numeric IDs (/users/1, /users/2, etc.). Any requester can enumerate all user IDs and access any user's full profile without ownership validation or authorization checks.
Evidence	Screenshot Test-2 – GET /users/1 returns the full profile of user 'Leanne Graham' including address, phone, geo-coordinates, and company details with Status 200 OK and no authorization.

Business Impact	Attackers can perform ID enumeration attacks to harvest all user profiles. In production, this is one of the most commonly exploited API vulnerabilities, enabling mass data exfiltration.
Remediation	Validate object ownership on every request. Ensure the requesting user can only access their own data. Use UUIDs instead of sequential IDs to prevent enumeration. Implement access control checks at the object level.

Finding 5: Missing Security Headers

Severity	**MEDIUM**
Endpoint(s)	All endpoints (observed via GET /users headers)
OWASP	API8:2023 – Security Misconfiguration
Observation	Response header analysis (Screenshot test1b) revealed the following missing security headers: Content-Security-Policy (CSP), X-Frame-Options, Strict-Transport-Security (HSTS), X-XSS-Protection, Permissions-Policy. The x-content-type-options header was present (value: nosniff) which is positive, however critical protection headers were absent.
Evidence	Screenshot test1b – Response headers show: x-content-type-options: nosniff, cache-control, server: cloudflare. Notable absence of Content-Security-Policy, X-Frame-Options, and Strict-Transport-Security headers.
Business Impact	Missing security headers leave API consumers and web clients vulnerable to clickjacking attacks (no X-Frame-Options), man-in-the-middle attacks (no HSTS), and cross-site scripting attacks (no CSP).
Remediation	Add the following headers to all API responses: Content-Security-Policy: default-src 'self', X-Frame-Options: DENY, Strict-Transport-Security: max-age=31536000; includeSubDomains, X-XSS-Protection: 1; mode=block.

Finding 6: Unrestricted Write Access (No Auth on POST)

Severity	**MEDIUM**
Endpoint(s)	POST /posts
OWASP	API2:2023 – Broken Authentication
Observation	A POST request to /posts was accepted and returned Status 201 Created with a new resource ID (id: 101) without requiring any authentication, authorization token, or session. The server accepted and processed the write operation from an anonymous caller.
Evidence	Screenshot Test-5 – POST /posts returns Status: 201 Created with id: 101. No Authorization header was included in the request, yet data creation was accepted.

Business Impact	In production, unauthenticated write access allows attackers to inject malicious content, spam records, corrupt data, or abuse business logic. This can lead to data integrity failures and reputational damage.
Remediation	Require authentication for all write operations (POST, PUT, PATCH, DELETE). Validate the identity and permissions of the caller before processing any data modification. Implement role-based access control (RBAC) to restrict write access to authorized users only.

7. Conclusion & Recommendations

The API security assessment of JSONPlaceholder identified six significant security risks that would represent serious vulnerabilities in a real production environment. The most critical issues are the complete absence of authentication controls and the exposure of sensitive user data without authorization checks.

The findings map directly to the OWASP API Security Top 10 framework, demonstrating that these are not theoretical risks but industry-recognized, commonly exploited vulnerabilities. Organizations deploying APIs with similar configurations would face significant risk of data breaches, regulatory penalties, and service disruption.

Priority Remediation Roadmap:

Priority	Timeline	Action	Findings Addressed
P1 – Critical	Immediate	Implement OAuth 2.0 / JWT authentication on all endpoints	Findings 1, 6
P1 – Critical	Immediate	Filter API responses to return only required fields (DTOs)	Finding 2
P2 – High	Within 1 week	Implement rate limiting and pagination on all list endpoints	Finding 3
P2 – High	Within 1 week	Validate object ownership on all ID-based endpoints	Finding 4
P3 – Medium	Within 2 weeks	Add missing security headers (CSP, HSTS, X-Frame-Options)	Finding 5

This assessment was conducted ethically and responsibly using only publicly available demo APIs. All findings are intended to educate and improve API security posture, not to exploit any systems.

End of Report